# PAL Manual

This manual starts with a small working model and then explains how PAL's pieces fit together. Use the API Guide for compact behavior summaries and the API Reference for signatures. A PDF copy of this Manual is also included for local/offline reading.

## 1. The model loop

A PAL model normally has three parts: **Setup** creates state, **Step** changes state, and **Draw** presents it. PAL does not impose a model class or a scheduler. You choose the update order and when to render.

```
import PythonAutomataLibrary as pal

@pal.njit(cache=True)
def Step(pop: pal.PopGrid):
    # Apply a synchronous increment to every site.
    for i in range(len(pop)):
        pop.Add(1, i)
    pop.Update()

def main():
    pop = pal.NewPopGrid((40, 40))
    for _ in range(10):
        Step(pop)
    print(pop.GetPop())

if __name__ == "__main__":
    main()
```

The example contains 1,600 sites and runs 10 increments per site, so the final total population is 16,000. `Add` accumulates changes; `Update` applies them together. That distinction is useful when one site's update must not affect its neighbors during the same timestep. The printed value is `16000`; every site contains `10`.

PAL uses **safe mode** by default. Develop and validate your model in this mode. If profiling shows that validation overhead matters, call `pal.FastMode()` before creating any PAL objects to remove selected performance-sensitive checks.

## 2. Choose the right representation

A `Grid` holds one typed value per site. An `AgentGrid` holds individually identifiable agents, optionally with per-agent properties. A `PopGrid` holds integer counts without individual identity. A `PDEgrid` holds a continuous scalar field and transport operations. These can be combined: for example, agents consume a PDE field while population counts accumulate in a separate PopGrid.

Use a tuple of dimensions when constructing a spatial grid. Negative dimensions mean periodic wrapping on that axis, so `(-40, 40)` wraps x but not y. `AgentGrid` also supports an empty dimension tuple for a nonspatial population.

### Choosing a representation by scale

Individual agents are useful when identity, history, or per-agent properties affect behavior. If agents are interchangeable within a site, a `PopGrid` replaces many handles with one integer count per site and can be much cheaper to update. Its optional `capacity` is a per-site population cap, not a cap on the total population. A `Grid` is appropriate for a site attribute that is not a population, such as a terrain type or a fixed mask. Use `PDEgrid` for a field whose values change continuously through diffusion or advection. A hybrid model can use different representations for different scales, but must explicitly define how state is exchanged between them.

Choose a representation based on what the model must preserve, not just on the number of sites. Converting individual agents to counts loses individual identity and history; representing a count as a continuous field can lose integer and stochastic behavior. Those are modeling assumptions, not implementation details.

# 3. Coordinates and neighborhoods

A grid has integer lattice sites and may also store agents at continuous positions. A **site index** `i` is an integer identifying a lattice cell; lattice coordinates such as `(x, y)` identify that cell on each axis. An agent's continuous coordinates are different: lattice site `(x, y)` spans continuous positions `[x, x+1) x [y, y+1)`, and its center is `(x+0.5, y+0.5)`. Thus an agent created or moved with `NewAgentSQ(x, y)` or `MoveSQ(..., x, y)` has integer site coordinates `XSQ=x`, `YSQ=y`, but continuous coordinates `X=x+0.5`, `Y=y+0.5`. An agent placed with `NewAgent` or `Move` instead keeps the supplied continuous coordinates while its `XSQ`/`YSQ` values identify the containing lattice site. In 2D, `ToI(x, y)` converts lattice coordinates to a linear site index, and `ItoX(i)` and `ItoY(i)` convert that index back to lattice coordinates. Do not confuse the linear site index, lattice coordinates, and continuous agent coordinates. Grid geometry is exposed as attributes such as `xDim`, `yDim`, `nDims`, and `wrapX`, not methods.

PAL provides `MooreHood`, `VonNeumannHood`, and `CircleHood`. They return relative integer offsets. `grid.Hood(hood, x, y)` maps those offsets to **lattice coordinates** (a scalar `x` in 1D, coordinate tuples in 2D/3D), wrapping periodic axes and omitting sites outside nonperiodic axes. It preserves the order and duplicates of the supplied offsets. `grid.Box(...)` instead iterates coordinates in a half-open rectangular region; in 2D it also yields `(x, y)` tuples, not linear indices. A box wider than a wrapped axis can also visit a site more than once. This distinction matters when using the results for indexing versus coordinate-based operations.

# 4. Individual agents

An agent is an integer handle owned by an `AgentGrid`, distinct from both its lattice site and its continuous position. Create it with `NewAgentSQ` for a lattice site or `NewAgent` for a continuous position. `NewAgentSQ` and `MoveSQ` accept either a linear site index or lattice coordinates; `NewAgent` and `Move` use continuous coordinates. An agent can move within a site without changing its lattice site. Remove it with `Dispose`.

`agents.All()` returns a snapshot of living handles. That allows creation and disposal during an iteration without changing the iteration's current membership. On a nonstackable grid, safe mode rejects attempts to place two agents at the same site. Use `isStackable=True` only when multiple occupancy is part of the model.

## Iterating nearby agents

Use `AgentsAt(x, y)` to visit the agents occupying one lattice site. For a continuous search around a point, `AgentsInRadius(rad, x, y)` yields `(agent, dx, dy, distSq)` in 2D. The displacement components account for periodic wrapping and `distSq` is squared distance. In 1D the tuple is `(agent, dx)`; in 3D it is `(agent, dx, dy, dz, distSq)`. Pass `exclude=agent` to omit an agent from its own neighborhood search.

Both iterators work in Python and inside `@pal.njit` loops. Python iteration materializes matching handles first; this does not permit structural modification of the grid during iteration. Safe mode checks for structural changes while iterating, whereas fast mode omits the check. For a loop that creates or disposes agents, iterate a snapshot from `agents.All()` instead.

For example, this Python-side search finds the agent across a periodic `x` boundary:

```
import PythonAutomataLibrary as pal

agents = pal.NewAgentGrid((-10, 10))
agent = agents.NewAgent(9.0, 2.0)
nearby = list(agents.AgentsInRadius(1.5, 0.0, 2.0))
assert len(nearby) == 1
found, dx, dy, distSq = nearby[0]
assert found == agent
assert dx == -1.0 and dy == 0.0 and distSq == 1.0
assert list(agents.AgentsInRadius(1.5, 0.0, 2.0, exclude=agent)) == []
```

## Agent birth/death timestep

`Examples/Agents/SaveLoadModel.py` is a complete agent-based model using a nonstackable 2D `AgentGrid`. Its compiled `Step` iterates `grid.All(shuffle=True)`, removes an agent on a death draw, and otherwise attempts reproduction on a second draw. To reproduce, it scans the Moore neighborhood, stores empty site indices in a reusable `IList`, and creates an agent at a randomly selected empty site.

The distinction between **snapshot iteration** and **current occupancy** matters here. `All()` fixes which agents get a turn at the beginning of the step; newborn agents do not act until the next step. But neighborhood occupancy is

queried when each parent acts, so births and deaths earlier in the same step can change later agents' options. Shuffling changes that order and therefore can change outcomes even with identical initial occupancy. This is a sequential stochastic agent update, unlike the simultaneous queued `PopGrid.Add`/`Update` pattern below.

The example's `Setup`, `Step`, and `Draw` functions are `@pal.njit(cache=True)` functions. The Python `main` loop owns the window, timestep, and checkpoints; compiled functions handle repeated model operations. Use the source example for the complete runnable program rather than duplicating its graphics and checkpoint lifecycle here.

## 5. Simultaneous population and field updates

Both `PopGrid` and `PDEgrid` distinguish current state from pending changes. Direct indexing changes current state immediately; `Add` queues a delta, and `Update` applies queued changes. `Reset` clears both current state and pending changes.

A `PDEgrid` also provides diffusion and advection. Set the timestep and spatial spacings with `SetTimeSpaceStep(dt, dx, dy, dz)` before transport operations. Cartesian, interface-based, masked, ADI, and radial diffusion variants are available; choose the method that matches the model's geometry and numerical assumptions. Cartesian boundaries may be uniform or vary across a face; wrapped axes are periodic instead. Validate timestep stability and boundary behavior in safe mode before optimizing.

### Advection versus diffusion example

`Examples/Diffusibles/ReactionDiffusion2D.py` compares two `PDEgrid` fields on the same 20 x 10 lattice. Each timestep sets a short vertical source at `x=10` in both fields. One field then calls `Advection(vx=0.01, vy=0.01, xMaxBC=0, yMaxBC=0)`; the other calls `DiffusionADI(0.01, xMaxBC=0, xMinBC=0)`. Each transport call is followed by its own `Update()`.

This makes the physical distinction visible: advection transports the field along a velocity, while diffusion spreads it down concentration gradients. Boundary concentrations also play different roles: diffusion couples them to the boundary gradient, while advection uses them only where velocity points into the domain; an advective outflow uses the interior upwind value. The explicit boundary arguments are part of the model, not cosmetic rendering settings. `Draw` maps each field's current values through `ColorScale` into separate pixel windows. As in the agent example, `Step` and `Draw` are compiled, while `main` manages windows and iteration.

The example is a qualitative comparison, **not** a convergence or mass-conservation test: it continually resets source cells and uses different boundary conditions for the two fields. For quantitative validation, remove the source, use equivalent boundary conditions, and check expected mass balance and grid/timestep convergence.

### A hand-checkable field gradient

`Examples/Diffusibles/Gradient2D.py` provides a simpler numerical check than a transport simulation. It assigns `grid[x, y] = y / grid.yDim` on a 10 x 10 `PDEgrid`. The centered finite differences at `(5, 5)` are therefore

```
import PythonAutomataLibrary as pal

field = pal.NewPDEgrid((10, 10))
for x in range(10):
    for y in range(10):
        field[x, y] = y / 10

dx = (field[6, 5] - field[4, 5]) / 2
dy = (field[5, 6] - field[5, 4]) / 2
assert abs(dx) < 1e-6
assert abs(dy - 0.1) < 1e-6
```

The field is constant in x and linear in y, so its expected discrete gradients are 0 and 0.1. The assertions allow floating-point storage and arithmetic error. This verifies indexing and a finite-difference calculation without introducing boundary conditions or timestep integration. `Examples/Diffusibles/Gradient2D.py` performs the same calculation inside `@pal.njit`.

### Numerical update order

The timestep order is part of the model. For example, queue all births and deaths in a `PopGrid` before calling `Update()` if changes must be simultaneous. Reading current counts while queuing deltas then uses the same starting population throughout the step. By contrast, assigning directly to `pop[x, y]` changes the value that subsequent calculations read. Mixing immediate assignments and queued changes is possible, but their order then changes the model.

For a `PDEgrid`, transport routines accumulate changes that become current at `Update()`. Check timestep stability and convergence by varying the timestep and grid spacing. Safe-mode checks do not replace numerical validation.

Consider two sites with populations 10 and 0. If a transfer moves 5 from site 0 to site 1, queue -5 and +5 before a single `Update()`; the result is (5, 5) and total population remains 10. Applying one change immediately would alter what later calculations read and can make an intended simultaneous update order-dependent.

For a small hand-calculated timestep, check the properties the model requires: counts stay nonnegative, internal transfers conserve total population, totals change only through explicit births, deaths, or boundary flux, and results are independent of site visitation order when the update is meant to be simultaneous.

## 6. Compiled model code

Decorate substantive computational functions with `@pal.njit(cache=True)` so compiled code can be reused across runs. PAL expands some model constructs before passing the function to Numba, including neighborhood iteration and source-aware safe-mode diagnostics. Annotate PAL arguments with their public types, such as `pal.AgentGrid` or `pal.PDEgrid`, for clarity and compiler support.

Keep model state in PAL objects and use `pal.Seed`, `pal.Random`, `pal.RandInt`, and `NewMultinomial` for reproducible PAL random sampling. Run a model in safe mode first. Fast mode removes selected checks; it does not make invalid indices, dead handles, or unstable numerical steps valid.

### Testing a timestep

Test a model's update function on a grid small enough to inspect by hand, including an edge or wrapped boundary rather than only interior sites. Use invariants appropriate to the object: closed diffusion conserves mass within floating-point tolerance; agent moves preserve valid live handles and occupancy rules; population updates satisfy the conservation and nonnegativity checks from section 5.

When a model uses both Python orchestration and compiled `@pal.njit` updates, test both paths; passing in Python does not establish compiled behavior. Include safe and fast runs in regression testing, but use safe-mode exceptions to diagnose invalid operations rather than expecting fast mode to detect them.

### Random streams and repeatability

`pal.Seed(seed)` initializes PAL's random stream. `pal.Random()` draws a uniform value and `pal.RandInt(max)` draws an integer from `[0, max)`. PAL random draws from Python and compiled PAL code consume the same stream, so **the order of calls matters**. Repeating a run requires the same seed, initial state, model parameters, and random-call order; merely reseeding halfway through a simulation does not reconstruct the prior model state.

For comparisons between two algorithms, seed each run separately and record the seed. If one implementation makes additional random draws, subsequent results can diverge even when both start from the same seed. External random libraries (such as NumPy's generators) have independent state and must be seeded or checkpointed separately.

## 7. Drawing and output

`StartPixWindow(xDim, yDim, scale=1, title='PAL', headless=False)` returns `(pix, window)`. Write packed RGB colors such as `0xFF0000` to `pix[x, y]`, then call `window.Update()` to publish the frame. `window.Save(path, block=True)` waits for the image to be written; use `window.Close()` to release resources. The pixel buffer also supports linear indices, slices, and writes from compiled code. For off-screen images, use `headless=True` (requires the relevant output dependencies). GIF recording uses `window.StartGif(path, delay=100)`, `window.AddGifFrame(block=False)`, and `window.StopGif()`; update the window before capturing each frame.

`StartOpenGLWindow` provides geometric 2D/3D drawing rather than a writable pixel lattice. Use its drawing object for shapes and scene controls; its window uses the same update, image/GIF output, and close lifecycle described above. `headless=True` is available for off-screen rendering but still requires a working OpenGL backend. See the API Guide or Reference for the drawing-method inventory.

Rendering and simulation are separate. A model can run without a window, draw every timestep, or draw only selected checkpoints.

## 8. Checkpointing and reproducibility

PAL's native-backed model objects support Python pickling, so a long-running model can save its state and resume without retaining native pointers from the original process. A checkpoint must include **all** state needed for continuation: every grid, relevant parameters, the current timestep, and any additional state managed by the model. Saving only one grid does not preserve a coupled simulation. Keep the code version and parameter configuration alongside the checkpoint so that results can be interpreted later.

A PAL object checkpoint does **not** serialize PAL's random stream. If exact stochastic continuation matters, the model must reconstruct the intended random state separately; external RNGs such as NumPy's are independent as well. Verify checkpoint behavior with a short uninterrupted run and a checkpoint-and-resume run rather than assuming object state alone reproduces the same future random draws.

A minimal checkpoint can be saved and resumed without opening a window:

```
import pickle
import PythonAutomataLibrary as pal

pop = pal.NewPopGrid((5,))
pop[1] = 10
pop.Add(7, 1)  # Pending change is part of the checkpoint.
state = {"pop": pop, "step": 3}
checkpoint = pickle.dumps(state)

restored = pickle.loads(checkpoint)
restored["pop"].Update()
assert restored["step"] == 3
assert restored["pop"][1] == 17
```

`Examples/Agents/SaveLoadModel.py` serializes a dictionary containing an `AgentGrid`, an `IList`, and the timestep, then restores all three before continuing. PAL object checkpoints do not include unrelated Python state or external RNG streams.

## 9. How PAL works

PAL separates its public Python interface from a native C core. Python constructors create native-backed model state, and thin Numba-compatible wrappers let compiled model code operate on that state without repeatedly returning to Python. Before Numba compilation, `pal.njit` transforms PAL-specific constructs such as `Hood` and `Box` iteration and adds source-aware diagnostics where annotations provide enough type information. The public Protocol classes describe the annotation/autocomplete surface while safe and fast concrete implementations provide runtime behavior.

This design favors fast repeated timesteps, but the first compiled call may include substantial Numba compilation time. Measure **cold compilation** separately from **steady-state execution**. Safe mode includes additional validation and source-aware diagnostics; fast mode removes some of that overhead. A change that improves one benchmark may regress another, so measure both modes on representative workloads.

## 10. Testing PAL

PAL's `Testing/` suite exercises public behavior across geometry and wrapping, Grid/IList/Multinomial/RNG operations, AgentGrid and PopGrid state changes, PDE transport and boundaries, compiled Python/Numba parity, visualization, integrated models, and historical regressions. Many tests use independent invariants or small reference calculations rather than reproducing PAL's implementation.

Run the full suite from the repository root in separate safe and fast processes:

```
PAL_TEST_MODE=safe python -m pytest -q Testing
PAL_TEST_MODE=fast python -m pytest -q Testing
```

PAL locks its process-wide mode at first construction, so safe and fast should not be combined in one Python process. Fast mode intentionally skips some validation tests. The CI workflow runs both modes on Linux, verifies all generated documentation artifacts are current, runs the API benchmark after successful tests, and separately runs representative examples in safe and fast modes on Windows. Headless OpenGL tests require the relevant graphics backend.

When adding or changing model code, start with the smallest test that expresses the intended public behavior, then run the affected test file before the full suite. Prefer conservation laws, coordinate identities, exact small examples, and Python/compiled agreement over tests that duplicate PAL's internal formulas.

# 11. Design principles: why PAL is shaped this way

This chapter makes explicit the tradeoffs that are easy to miss when reading the API as a list of methods. Some choices are direct consequences of the implementation; others are engineering priorities reflected in the public contract. They should not be mistaken for claims that every alternative was exhaustively benchmarked.

## 11.1 Keep model logic in Python, put repeated work in compiled kernels

PAL is designed for models whose timestep performs many small operations: inspect a neighborhood, update state, move agents, accumulate a field, or sample a random event. Python is productive for configuring experiments, loading data, controlling the simulation loop, and drawing. But calling Python for every cell or agent can dominate the cost of a large model.

PAL therefore separates orchestration from the inner loop. A normal Python function can create objects, open a window, choose parameters, and decide how many timesteps to run. Functions decorated with `@pal.njit(cache=True)` express the repeated computation. The cache can avoid recompiling the same specialization on later runs, while the first call may still pay a compilation cost.

This division is a recommendation, not a requirement that every function be compiled. Start with clear Python code, identify the repeated work, and move the computational kernel into compiled code when that is useful. Keep file I/O, interactive controls, and other orchestration outside compiled functions unless the public API explicitly supports them.

## 11.2 Use a small set of representations with distinct meanings

PAL deliberately distinguishes a typed `Grid`, an `AgentGrid`, a `PopGrid`, and a `PDEgrid`. These are not interchangeable containers with different names. Each representation encodes assumptions about what the model needs to remember:

- A `Grid` stores a value at each lattice site. Use it for attributes such as terrain, occupancy masks, or local state that is not itself a collection of individuals. - An `AgentGrid` preserves individual identity. Agents can have different positions, histories, properties, and fates. - A `PopGrid` stores integer counts. It is suited to interchangeable individuals when the model needs the number at each site, but not a handle for each one. - A `PDEgrid` stores a scalar field and supplies numerical transport operations such as diffusion and advection.

The point is not to force every model into one abstraction. It is to make the meaning of state explicit. A count grid avoids per-individual bookkeeping, but cannot answer which individual moved or what its personal history was. A continuous field can represent concentration smoothly, but it does not preserve integer identity or discrete events. A hybrid model is often appropriate, provided the conversions between representations are part of the scientific model rather than accidental implementation details.

## 11.3 Make update timing visible

A simulation's update order is part of its mathematics. PAL does not hide every update behind an automatic scheduler because different models need different timing semantics. A synchronous cellular update, a sequential agent sweep, and a numerical field solve are not the same operation.

For `PopGrid` and `PDEgrid`, queued changes and `Update()` provide a transaction-like boundary: calls that accumulate deltas do not immediately become the current state. This lets a model calculate changes from one shared starting state and then commit them together. Direct indexing, by contrast, changes current state immediately. Mixing the two is permitted, but the order then matters and must be deliberate.

An `AgentGrid` has a different contract. `All()` returns a snapshot of the living handles at the time it is called, so adding or removing agents does not change which handles are in that snapshot. It does not freeze the entire world: occupancy queries performed later still see the current grid, including births and deaths that have already happened. This is useful for a sequential agent update, but it is not a synchronous update. If all agents must act from the same old state, store the necessary old values separately and commit changes in a second phase.

## 11.4 Make dimensionality explicit but keep the API unified

Most model concepts-grids, neighborhoods, movement, and fields-exist in one, two, or three spatial dimensions. Requiring a separate class and method family for every dimension would multiply the API surface and make examples harder to transfer between models. PAL instead uses dimension-aware objects and coordinate tuples, with shared geometry metadata such as `nDims`, `xDim`, `yDim`, `zDim`, and wrapping flags.

The coordinate shape still matters. In one dimension a site can be represented by one integer; in two and three dimensions, coordinate tuples identify the axes. PAL retains explicit site-index operations where they are useful, but APIs that mean continuous position remain distinct from APIs that mean lattice position. This prevents a linear index, a lattice coordinate, and a continuous location from being silently treated as the same thing.

A dimensions tuple also communicates the shape of a lattice. A negative extent requests wrapping on that axis; for example, `(-40, 40)` wraps x but not y. This is compact, but it means that the sign of a dimension is configuration, not a request for a negative-sized array. When reading a model, always check both the absolute extents and the wrap flags implied by the signs.

## 11.5 Distinguish lattice coordinates from continuous positions

A lattice site is discrete; an off-lattice agent position is continuous. PAL keeps both concepts because many models need both: an agent may move continuously while still interacting with fields stored at lattice sites.

For a lattice site `(x, y)`, the associated unit cell occupies `[x, x+1) x [y, y+1)`. Its center is `(x+0.5, y+0.5)`. Creating an agent with `NewAgentSQ(x, y)` places it at the site center; creating it with `NewAgent(x, y)` uses the continuous coordinates supplied. The resulting `XSQ`/`YSQ` values identify the containing site, while `X`/`Y` retain continuous position.

This distinction affects neighborhood searches, visualization, coupling to fields, and boundary behavior. Use site-based movement when the model is lattice-based. Use continuous movement when sub-site displacement matters. Do not round continuous positions manually unless that rounding is part of the model; use the documented conversion or site properties.

## 11.6 Separate relative neighborhoods from their application

`MooreHood`, `VonNeumannHood`, and `CircleHood` describe relative integer offsets. A hood is reusable geometry: it is not itself a list of absolute sites around a particular location. Applying `grid.Hood(hood, x, y)` translates those offsets to lattice coordinates, applies wrapping, and omits coordinates outside nonperiodic boundaries.

This separation makes neighborhood definitions independent of the grid location and allows PAL to expand neighborhood loops in compiled functions. It also exposes an important detail: wrapping may map multiple offsets to the same site, and the API preserves offset order and duplicates. If duplicate visits would double-count a contribution, choose a neighborhood and domain size that avoid them or explicitly deduplicate according to the model's intended semantics.

`Box` has a different purpose: it describes a half-open rectangular region. The upper bound is excluded, so `Box(0, 4)` visits 0, 1, 2, and 3 along that axis. Neighborhood offsets are relative; Box bounds describe a region. Confusing the two produces off-by-one errors and boundary mistakes.

## 11.7 Make safety a process-level choice

PAL's safe mode is the default. It adds validation and diagnostics around selected operations, helping catch invalid indices, illegal occupancy changes, and other contract violations close to their source. This is especially valuable while developing a new model, when an error may otherwise appear later as a corrupted result.

Fast mode removes selected checks for performance-sensitive paths. It does not turn invalid operations into valid ones, and it does not make a numerically unstable scheme stable. PAL locks the process-wide mode when the first PAL model or window is constructed; call `pal.FastMode()` before any such construction. Because the choice is

process-wide, compare safe and fast behavior in separate processes rather than trying to switch modes midway through a run.

This is an explicit tradeoff: predictable diagnostics during development, with a deliberate route to reduce selected overhead after profiling. Safe mode is not a proof of scientific correctness. A program can use every API correctly and still encode the wrong equations or assumptions.

### 11.8 Use a native core behind a Python-facing API

The public Python API is intended to remain readable and compatible with Numba-compiled model code. The native C core owns important storage and kernels, while Python and Numba-facing wrappers expose that functionality to ordinary Python and compiled functions. The wrapper layer is therefore API glue, not a second independent implementation of the model state.

This separation lets PAL combine Python usability with native-backed data structures and compiled loops. It also creates obligations: wrapper signatures, argument names, indexing behavior, safe-mode checks, and Python/compiled behavior must agree. A mismatch between a Python path and a compiled path is a correctness defect even if each path appears plausible in isolation.

The public Protocol declarations provide the annotation and autocomplete surface. They should describe the supported public contract, but a declaration alone is not proof of runtime behavior. The implementation and executable tests must agree with the published contract. When changing an API, update its behavior, type-facing declarations, reference material, examples, and regression tests together.

### 11.9 Prefer explicit state changes and inspectable contracts

PAL uses explicit operations such as `Add`, `Update`, `Reset`, `Move`, and `Dispose` rather than relying on hidden automatic mutation. The names make important state transitions visible in a timestep. For example, a queued delta is not the same as a committed population, and a moved agent is not the same as an agent whose handle has been disposed.

This explicitness requires the model author to understand lifecycle and update order, but it makes the simulation easier to reason about and test. For stateful models, clarity about when state changes is more valuable than hiding those changes behind a convenient-looking abstraction.

### 11.10 Why safe/fast and Python/compiled parity are tested separately

A model can fail in more than one way: an operation can violate the API contract; the Python and compiled paths can disagree; or both can behave consistently while the scientific model is wrong. PAL's tests try to separate these failure classes.

Safe and fast modes are tested in separate processes because mode selection is locked at construction. Python and compiled paths are compared because the public API is intended to work in both settings. Small exact examples and independent invariants-such as conservation, coordinate identities, and known gradients-are preferred where possible because they can reveal a shared implementation mistake that a test duplicating the implementation's formula might miss.

Integrated examples then exercise combinations of components under realistic control flow. No single test layer is sufficient: focused tests localize defects, parity tests catch execution-path differences, invariants check mathematical properties, and full examples reveal integration problems.

## 12. Performance: measure the right thing

### 12.1 Cold compilation is not steady-state runtime

The first call to a compiled function may include Numba compilation and specialization. Later calls can be much faster, particularly when caching is enabled. Timing one first call and reporting it as the simulation's runtime mixes two different costs.

For a useful benchmark, measure cold compilation in a fresh process and steady-state runtime over repeated timesteps separately. Use the same model state, workload, seed, and operation count for each comparison. Report medians over repeated runs rather than relying on one timing. If comparing safe and fast modes, use separate processes because the mode is fixed at construction.

### 12.2 Optimize the actual workload

A faster microbenchmark does not guarantee a faster full model. An optimization may help a large agent sweep but hurt a small grid, or reduce steady-state time while increasing compilation cost. Before changing implementation, define the workload that matters: number of sites or agents, number of timesteps, neighborhood size, mutation rate, transport method, and mode.

Use a before/after benchmark with fresh-process cold compilation and repeated hot runtime. Keep workload sizes identical, check outputs for equivalence, and investigate any reproducible regression. Treat tiny differences as noise until repeated measurements show otherwise. Prefer simpler correct code when a performance gain is not robust.

### 12.3 Know what fast mode cannot fix

Fast mode can reduce selected validation overhead; it cannot eliminate the fundamental cost of updating millions of state elements, choosing an inefficient representation, doing unnecessary neighborhood work, or solving an unstable numerical scheme. Before optimizing low-level code, ask whether the model's algorithm is doing unnecessary work. Replacing individually represented interchangeable agents with counts, reusing a neighborhood list, reducing repeated conversions, or drawing less often can have a larger impact than a local kernel tweak.

Do not optimize by removing a check merely because it appears in a profile. First establish that the check is responsible for a meaningful fraction of runtime, that the faster behavior is safe for the intended workload, and that tests cover the affected contract.

## 13. Validation: prove the model, not just the program

### 13.1 Start with a hand-calculable case

Before running a large simulation, reduce the domain and timestep count until the expected answer can be computed manually. A two-site transfer can establish whether queued population changes conserve the total. A linear field can establish whether indexing and finite differences produce the expected gradient. A tiny periodic grid can show exactly which neighbor lies across a boundary.

Small examples are powerful because they turn vague visual plausibility into an assertion with a known answer. Keep such examples as regression tests after the model grows.

### 13.2 Test invariants and edge cases

Choose invariants from the scientific model. For a closed diffusion problem, total mass should be conserved within a stated numerical tolerance. For an internal population transfer, the total should remain constant and counts should stay nonnegative. For an agent model, every live handle should refer to a living agent and occupancy should obey the stacking rule. For geometry, conversions should round-trip and wrapping should return the expected coordinates.

Test the cases most likely to break assumptions: the first and last site, wrapped and nonwrapped axes, empty neighborhoods, a single agent, zero or maximal allowed counts, and updates that add and remove state in the same timestep. A model that works only away from boundaries has not yet validated its geometry.

### 13.3 Separate API correctness from scientific validity

Safe-mode checks and passing unit tests show that operations respect tested contracts. They do not establish that the chosen growth law, diffusion coefficient, update schedule, stochastic distribution, or boundary condition is scientifically justified. Validate the numerical method against known solutions where possible; test convergence as grid spacing and timestep change; and compare stochastic results across independent seeds.

Document assumptions alongside the model: whether agents update sequentially or synchronously, whether boundaries are reflecting, fixed, or periodic, whether rates are per unit time or per step, and whether counts or fields represent densities or totals. These assumptions determine what the simulation means.

## 14. Where to go next

Use the Cheatsheet for short code patterns, the API Guide for behavioral contracts, and the generated API Reference for current signatures. `Examples/` contains complete models; the Testing guide describes regression coverage and how to run the suite.